# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

| | | | |
|---|---|---|---|
| **Course Code:** | CSA0620 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | CO1 – Precision (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 1 – Application Based Questions |
| **Weightage:** | 40% of CIA | **Total Marks:** | 100 |
| **CO1 Statement:** | *Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)* | | |
| **Student Name:** | A. DEKKSHITHA | **Reg. No.:** | 192572133 |
| **Section / Batch:** | | **Date:** | 28.07.2026 |

**Instructions to Students**

- Answer the following question. It carries 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

| Question Type | Focus Area | Questions |
|---|---|---|
| **Application-Based Questions** | Apply Master Theorem and asymptotic analysis to real-world scenarios | Q1 |

### SECTION A – APPLICATION BASED QUESTIONS

## Q1 . Fraud Detection in Financial Systems

A fraud detection system processes transactions using the recurrence $T(n) = T(n-1) + n^2$. Solve the recurrence using substitution method and determine the time complexity. Also analyze the space complexity.

**Code of Conduct**

*I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20% | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts | 30% | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20% | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| Accuracy of Solution | 20% | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| Clarity | 10% | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

## Marks Evaluation:

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20% | | | | |
| Application of Concepts | 30% | | | | |
| Problem-Solving Approach | 20% | | | | |
| Accuracy of Solution | 20% | | | | |
| Clarity | 10% | | | | |
| **Total Marks (100):** | | | | | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| Name: | Name: | Name: |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# Q.1 Fraud Detection in Financial Systems

**CODE :**

```
import time
import tracemalloc
def fraud_detection_recursive(n):
    if n == 0:
        return 0
    return fraud_detection_recursive(n - 1) + (n * n)
def fraud_detection_formula(n):
    return (n * (n + 1) * (2 * n + 1)) // 6
def display_steps(n):
    print("\nExpansion using Substitution Method")
    print("-----------------------------------")
    total = 0
    for i in range(1, n + 1):
        total += i * i
        print(f"T({i}) = T({i-1}) + {i}² = {total}")
    return total
def main():
    print("=" * 60)
    print(" FRAUD DETECTION IN FINANCIAL SYSTEMS ")
    print("=" * 60)
    n = int(input("Enter number of transactions (n): "))
    tracemalloc.start()
    start = time.perf_counter()
    recursive_result = fraud_detection_recursive(n)
    end = time.perf_counter()
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print()
    display_steps(n)
    formula_result = fraud_detection_formula(n)
    print("\n-----------------------------------")
    print("Results")
    print("-----------------------------------")
    print("Recursive Result :", recursive_result)
    print("Formula Result   :", formula_result)
```

```
    if recursive_result == formula_result:
        print("Verification     : Correct ✓")
    else:
        print("Verification     : Incorrect")
    print("\nExecution Time")
    print("----------------")
    print(f"{(end-start):.8f} seconds")
    print("\nMemory Usage")
    print("----------------")
    print(f"Current Memory : {current} bytes")
    print(f"Peak Memory    : {peak} bytes")
    print("\nComplexity Analysis")
    print("-----------------------")
    print("Time Complexity  : O(n)")
    print("Space Complexity : O(n) (recursive call stack)")
    print("\nClosed Form")
    print("-----------------------")
    print("T(n) = n(n+1)(2n+1)/6")
if __name__ == "__main__":
    main()
```

**OUTPUT :**

```
Output
============================================================
 FRAUD DETECTION IN FINANCIAL SYSTEMS
============================================================
Enter number of transactions (n): 5


Expansion using Substitution Method
-----------------------------------
T(1) = T(0) + 1² = 1
T(2) = T(1) + 2² = 5
T(3) = T(2) + 3² = 14
T(4) = T(3) + 4² = 30
T(5) = T(4) + 5² = 55

-----------------------------------
Results
-----------------------------------
Recursive Result : 55
Formula Result   : 55
```

**Output**

```
Results
-----------------------------------
Recursive Result : 55
Formula Result   : 55
Verification     : Correct ✓

Execution Time
---------------
0.00000676 seconds

Memory Usage
---------------
Current Memory : 0 bytes
Peak Memory    : 0 bytes

Complexity Analysis
----------------------
Time Complexity  : O(n)
Space Complexity : O(n) (recursive call stack)
```

```
Complexity Analysis
-----------------------
Time Complexity  : O(n)
Space Complexity : O(n) (recursive call stack)

Closed Form
-----------------------
T(n) = n(n+1)(2n+1)/6

=== Code Execution Successful ===
```

# Execution :

main.py

```
import time
import tracemalloc
def fraud_detection_recursive(n):
    if n == 0:
        return 0
    return fraud_detection_recursive(n - 1) + (n * n)
def fraud_detection_formula(n):
    return (n * (n + 1) * (2 * n + 1)) // 6
def display_steps(n):
    print("\nExpansion using Substitution Method")
    print("-----------------------------------")
    total = 0
    for i in range(1, n + 1):
        total += i * i
        print(f"T({i}) = T({i-1}) + {i}² = {total}")
    return total
def main():
    print("=" * 60)
    print(" FRAUD DETECTION IN FINANCIAL SYSTEMS ")
```

Output

```
============================================================
 FRAUD DETECTION IN FINANCIAL SYSTEMS
============================================================
Enter number of transactions (n): 5


Expansion using Substitution Method
-----------------------------------
T(1) = T(0) + 1² = 1
T(2) = T(1) + 2² = 5
T(3) = T(2) + 3² = 14
T(4) = T(3) + 4² = 30
T(5) = T(4) + 5² = 55

-----------------------------------
Results
-----------------------------------
Recursive Result : 55
Formula Result   : 55
```

main.py

```
    print("=" * 60)
    n = int(input("Enter number of transactions (n): "))
    tracemalloc.start()
    start = time.perf_counter()
    recursive_result = fraud_detection_recursive(n)
    end = time.perf_counter()
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print()
    display_steps(n)
    formula_result = fraud_detection_formula(n)
    print("\n-----------------------------------")
    print("Results")
    print("-----------------------------------")
    print("Recursive Result :", recursive_result)
    print("Formula Result   :", formula_result)
    if recursive_result == formula_result:
        print("Verification     : Correct ✓")
    else:
```

Output

```
-----------------------------------
Results
-----------------------------------
Recursive Result : 55
Formula Result   : 55
Verification     : Correct ✓

Execution Time
---------------
0.00000676 seconds

Memory Usage
---------------
Current Memory : 0 bytes
Peak Memory    : 0 bytes

Complexity Analysis
-----------------------
Time Complexity  : O(n)
```

Programiz Python Online Compiler — Programiz PRO ›

main.py

```
        print("Verification     : Correct ✓")
    else:
        print("Verification     : Incorrect")
    print("\nExecution Time")
    print("---------------")
    print(f"{(end-start):.8f} seconds")
    print("\nMemory Usage")
    print("---------------")
    print(f"Current Memory : {current} bytes")
    print(f"Peak Memory    : {peak} bytes")
    print("\nComplexity Analysis")
    print("-----------------------")
    print("Time Complexity  : O(n)")
    print("Space Complexity : O(n) (recursive call stack)")
    print("\nClosed Form")
    print("-----------------------")
    print("T(n) = n(n+1)(2n+1)/6")
if __name__ == "__main__":
    main()
```

Output

```
Execution Time
---------------
0.00000676 seconds

Memory Usage
---------------
Current Memory : 0 bytes
Peak Memory    : 0 bytes

Complexity Analysis
-----------------------
Time Complexity  : O(n)
Space Complexity : O(n) (recursive call stack)

Closed Form
-----------------------
T(n) = n(n+1)(2n+1)/6

=== Code Execution Successful ===
```